

GUION DOCENTE — CLASE 3

Importación CSV + Capa de Servicios + Consultas OLAP con SQLAlchemy ORM

Duración estimada: 3 horas

Modalidad: práctica guiada

Proyecto: Dashboard SaaS Café Andino

Carácter de la clase: incremental

Punto de partida: Clase 1 + Clase 2 ya integradas

Nivel: intermedio — Programación Web + Bases de Datos + Análítica Comercial

1. Sentido de la clase

En la Clase 1 se construyó la estructura del sistema web: Flask, Blueprints, login, templates y layout SaaS. En la Clase 2 se conectó el sistema con PostgreSQL mediante SQLAlchemy ORM, se crearon los modelos y se aplicaron migraciones. En la Clase 3 se trabajará el proceso que convierte un archivo CSV en información útil para el sistema: lectura, limpieza, inserción en base de datos y consultas agregadas tipo OLAP.

2. Objetivo general

Importar un archivo CSV de ventas comerciales hacia PostgreSQL usando SQLAlchemy ORM, organizar la lógica en una capa de servicios y construir consultas agregadas para obtener información analítica que luego será utilizada en el dashboard final.

3. Objetivos específicos

Al finalizar la clase, el estudiante será capaz de:

- Comprender el flujo CSV → Python → ORM → PostgreSQL.
- Leer un archivo CSV desde una aplicación Flask.
- Validar y transformar datos antes de insertarlos en la base de datos.
- Controlar duplicados mediante una clave de referencia del CSV.
- Separar la lógica de negocio en `services.py`.
- Construir consultas agregadas con SQLAlchemy ORM.
- Aplicar `SUM`, `AVG`, `COUNT`, `GROUP BY`, `ORDER BY` y `LIMIT`.
- Interpretar consultas tipo OLAP para ventas, productos, clientes y sucursales.
- Exponer resultados en una vista simple o como JSON.

4. Resultado final de la clase

Al terminar la sesión, el proyecto deberá tener:

- Tabla `ventas` poblada con datos importados desde CSV.
- Función de importación funcionando.
- Control de duplicados por `id_venta_csv`.
- Capa `services.py` con funciones reutilizables.
- Consultas agregadas comerciales.
- Ruta protegida para ejecutar importación o visualizar resultados.
- Vista básica mostrando indicadores analíticos.

5. Idea central para explicar al estudiante

“En la Clase 2 creamos la estructura de la base de datos. Hoy vamos a llenarla con datos reales y a consultar esa información para empezar a construir inteligencia comercial.”

6. Flujo conceptual de la clase

```
CSV de ventas
  ↓
Lectura con Python
  ↓
Limpieza y conversión de tipos
  ↓
Creación de objetos Venta
  ↓
Inserción con SQLAlchemy ORM
  ↓
PostgreSQL
  ↓
Consultas agregadas
  ↓
Indicadores para dashboard
```

7. Archivos que se abrirán en orden

1. `app/models.py`
 2. `app/data/services.py`
 3. `app/data/routes.py`
 4. `app/__init__.py`
 5. `app/dashboard/routes.py`
 6. `app/templates/dashboard.html` o template equivalente
 7. `dataset_cafe_andino.csv`
 8. `.env`
 9. `run.py`
-

8. Archivos que se modifican en esta clase

Se modifican directamente

- `app/data/services.py`
- `app/data/routes.py`
- `app/dashboard/routes.py`
- `app/templates/dashboard.html` o template de resumen
- `app/__init__.py` si falta registrar el Blueprint de data

Se revisan, pero no se rediseñan

- `app/models.py`
- `app/config.py`

No se trabaja todavía

- `static/js/dashboard.js`
 - `static/js/datatables.js`
 - Chart.js avanzado
 - DataTables avanzado
 - diseño visual final de los dashboards
-

9. Estructura de la clase

Bloque	Tema	Tiempo
1	Repaso de Clase 1 y Clase 2	10 min

Bloque	Tema	Tiempo
2	Presentación del flujo CSV → BD → OLAP	15 min
3	Revisión del modelo <code>Venta</code>	20 min
4	Lectura del CSV y análisis de columnas	20 min
5	Creación de funciones auxiliares de limpieza	25 min
6	Importación del CSV con ORM	40 min
7	Ruta protegida de importación	20 min
8	Consultas agregadas tipo OLAP	45 min
9	Integración mínima con dashboard	25 min
10	Práctica final, validación y cierre	20 min

BLOQUE 1 — Repaso de Clase 1 y Clase 2

Tiempo: 10 min

Qué mostrar

Ejecutar el proyecto y mostrar:

- login funcional.
- dashboard protegido.
- conexión activa a PostgreSQL.
- tablas `usuarios` y `ventas` creadas.

Qué decir

“Hasta ahora el sistema tiene estructura, seguridad y base de datos. Sin embargo, todavía no tiene datos comerciales. Hoy cargaremos el dataset y construiremos las primeras consultas de negocio.”

Preguntas rápidas

- ¿Qué archivo define los modelos ORM?
- ¿Qué comando aplica las migraciones?
- ¿Qué representa una clase ORM?
- ¿Por qué no debemos guardar contraseñas en texto plano?

BLOQUE 2 — Presentación del flujo CSV → BD → OLAP

Tiempo: 15 min

Explicación docente

Un dashboard profesional no debe leer directamente el CSV cada vez que se abre. Lo correcto es cargar el CSV a una base de datos y luego consultar PostgreSQL mediante ORM.

Comparación

Enfoque básico	Enfoque profesional
Leer CSV directo en cada vista	Cargar CSV a PostgreSQL
Datos temporales	Datos persistentes
Difícil escalar	Más ordenado y consultable
Sin control de duplicados	Control por clave única
Lógica mezclada	Lógica separada en servicios
## Mensaje clave	
> "El CSV es la fuente inicial. PostgreSQL será la fuente persistente del sistema."	

BLOQUE 3 — Revisión del modelo Venta

Tiempo: 20 min

Archivo

`app/models.py`

Qué revisar

Verificar que el modelo Venta contenga los campos necesarios del CSV.

Código de referencia

```
class Venta(db.Model):
    __tablename__ = "ventas"
    id = db.Column(db.Integer, primary_key=True)
    id_venta_csv = db.Column(db.String(50), unique=True, nullable=False)
    fecha_venta = db.Column(db.Date)
    hora_venta = db.Column(db.String(20))
    canal_venta = db.Column(db.String(50))
    sucursal = db.Column(db.String(100))
    region_sucursal = db.Column(db.String(100))
    id_cliente = db.Column(db.String(50))
    nombre_cliente = db.Column(db.String(150))
    tipo_cliente = db.Column(db.String(80))
    segmento_cliente = db.Column(db.String(80))
    ciudad_cliente = db.Column(db.String(100))
    zona_cliente = db.Column(db.String(100))
    nombre_producto = db.Column(db.String(150))
    categoria_producto = db.Column(db.String(100))
    linea_producto = db.Column(db.String(100))
    presentacion = db.Column(db.String(100))
    origen_cafe = db.Column(db.String(100))
    tipo_tueste = db.Column(db.String(80))
    precio_unitario = db.Column(db.Float)
    cantidad = db.Column(db.Integer)
    descuento_porcentaje = db.Column(db.Float)
    total_bruto = db.Column(db.Float)
    total_descuento = db.Column(db.Float)
    total_netto = db.Column(db.Float)
    fecha_carga = db.Column(db.DateTime, default=datetime.utcnow)
```

Explicación de campos clave

- `id_venta_csv` : identifica de forma única cada venta del CSV.
- `fecha_venta` : permite análisis temporal.
- `canal_venta` : permite analizar ventas por canal.
- `sucursal` y `region_sucursal` : permiten análisis geográfico/comercial.
- `tipo_cliente` y `segmento_cliente` : permiten análisis de clientes.
- `categoria_producto`, `linea_producto`, `origen_cafe`, `tipo_tueste` : permiten análisis de productos.
- `total_netto` : principal métrica comercial.
- `cantidad` : métrica de volumen.

Mensaje docente

“El modelo Venta es la traducción del CSV al mundo ORM.”

BLOQUE 4 — Lectura del CSV y análisis de columnas

Tiempo: 20 min

Archivo

dataset_cafe_andino.csv

Qué mostrar

Abrir el CSV y observar columnas. Columnas esperadas:

```
id_venta, fecha_venta, hora_venta, canal_venta, sucursal, region_sucursal,
id_cliente, nombre_cliente, tipo_cliente, segmento_cliente, ciudad_cliente,
zona_cliente, nombre_producto, categoria_producto, linea_producto,
presentacion, origen_cafe, tipo_tueste, precio_unitario, cantidad,
descuento_porcentaje, total_bruto, total_descuento, total_netto
```

Qué explicar

No todos los valores del CSV llegan listos para insertarse. Algunos deben convertirse:

- fechas → `date`
- cantidades → `int`
- importes → `float`
- textos vacíos → `None` o cadena limpia

Mensaje docente

“Antes de guardar un dato, debemos revisar si está en el tipo correcto.”

BLOQUE 5 — Funciones auxiliares de limpieza

Tiempo: 25 min

Archivo

app/data/services.py

Objetivo

Crear funciones pequeñas para limpiar valores antes de insertarlos.

Código de referencia

```
import csv
from datetime import datetime
from sqlalchemy import func, extract
from app import db
from app.models import Venta

def limpiar_texto(valor):
    if valor is None:
        return None
    valor = str(valor).strip()
    return valor if valor != "" else None

def convertir_float(valor):
    try:
        if valor is None or valor == "":
            return 0.0
        return float(str(valor).replace(",", "."))
    except ValueError:
        return 0.0

def convertir_int(valor):
    try:
        if valor is None or valor == "":
            return 0
        return int(float(valor))
    except ValueError:
        return 0

def convertir_fecha(valor):
    if not valor:
        return None
    formatos = ["%Y-%m-%d", "%d/%m/%Y", "%m/%d/%Y"]
```



```
for formato in formatos:
    try:
        return datetime.strptime(valor, formato).date()
    except ValueError:
        continue
return None
```

Qué explicar

- `limpiar_texto()` evita espacios innecesarios.
- `convertir_float()` prepara importes comerciales.
- `convertir_int()` prepara cantidades.
- `convertir_fecha()` permite manejar varios formatos de fecha.

Pregunta para estudiantes

- ¿Qué problema podría ocurrir si intentamos guardar una fecha como texto?

BLOQUE 6 — Importación del CSV con ORM

Tiempo: 40 min

Archivo

`app/data/services.py`

Objetivo

Leer el CSV, crear objetos `Venta` y guardarlos en PostgreSQL.

Código de referencia

```
def importar_ventas_csv(ruta_archivo):
    creados = 0
    omitidos = 0
    errores = 0

    with open(ruta_archivo, mode="r", encoding="utf-8-sig", newline="") as
archivo:
        reader = csv.DictReader(archivo)

        for fila in reader:
```

```

try:
    id_venta = limpiar_texto(fila.get("id_venta"))

    if not id_venta:
        errores += 1
        continue

    existe = Venta.query.filter_by(id_venta_csv=id_venta).first()
    if existe:
        omitidos += 1
        continue

    venta = Venta(
        id_venta_csv=id_venta,
        fecha_venta=convertir_fecha(fila.get("fecha_venta")),
        hora_venta=limpiar_texto(fila.get("hora_venta")),
        canal_venta=limpiar_texto(fila.get("canal_venta")),
        sucursal=limpiar_texto(fila.get("sucursal")),
        region_sucursal=limpiar_texto(fila.get("region_sucursal")),
        id_cliente=limpiar_texto(fila.get("id_cliente")),
        nombre_cliente=limpiar_texto(fila.get("nombre_cliente")),
        tipo_cliente=limpiar_texto(fila.get("tipo_cliente")),

segmento_cliente=limpiar_texto(fila.get("segmento_cliente")),
        ciudad_cliente=limpiar_texto(fila.get("ciudad_cliente")),
        zona_cliente=limpiar_texto(fila.get("zona_cliente")),
        nombre_producto=limpiar_texto(fila.get("nombre_producto")),

categoria_producto=limpiar_texto(fila.get("categoria_producto")),
        linea_producto=limpiar_texto(fila.get("linea_producto")),
        presentacion=limpiar_texto(fila.get("presentacion")),
        origen_cafe=limpiar_texto(fila.get("origen_cafe")),
        tipo_tueste=limpiar_texto(fila.get("tipo_tueste")),

precio_unitario=convertir_float(fila.get("precio_unitario")),
        cantidad=convertir_int(fila.get("cantidad")),

descuento_porcentaje=convertir_float(fila.get("descuento_porcentaje")),
        total_bruto=convertir_float(fila.get("total_bruto")),

total_descuento=convertir_float(fila.get("total_descuento")),
        total_neto=convertir_float(fila.get("total_neto"))
    )

    db.session.add(venta)
    creados += 1

except Exception:

```

```

        errores += 1

    db.session.commit()

    return {
        "creados": creados,
        "omitidos": omitidos,
        "errores": errores
    }

```

Explicación paso a paso

1. Se abre el CSV.
2. `csv.DictReader` lee cada fila como diccionario.
3. Se obtiene `id_venta`.
4. Si el ID no existe, se considera error.
5. Si el ID ya está en base de datos, se omite.
6. Si es nuevo, se crea un objeto `Venta`.
7. El objeto se agrega a la sesión ORM.
8. Al final se ejecuta `commit()`.

Mensaje clave

“Cada fila del CSV se convierte en un objeto Python y luego en una fila de PostgreSQL.”

Advertencia docente

No ejecutar `commit()` en cada fila si el dataset es grande. Para clase se puede hacer, pero es mejor hacerlo al final por eficiencia.

BLOQUE 7 — Ruta protegida de importación

Tiempo: 20 min

Archivo

`app/data/routes.py`

Código de referencia

```

import os
from flask import Blueprint, jsonify, current_app

```

```

from flask_login import login_required
from app.data.services import importar_ventas_csv

data_bp = Blueprint("data", __name__, url_prefix="/data")

@data_bp.route("/importar-ventas")
@login_required
def importar_ventas():
    ruta_csv = os.path.join(current_app.root_path, "..",
"dataset_cafe_andino.csv")
    resultado = importar_ventas_csv(ruta_csv)
    return jsonify(resultado)

```

Registrar Blueprint si falta

Archivo: `app/__init__.py`

```

from app.data.routes import data_bp
app.register_blueprint(data_bp)

```

Qué explicar

- La ruta está protegida con `login_required`.
- Solo usuarios autenticados pueden importar datos.
- La importación devuelve un JSON con resumen.

Práctica

1. Iniciar sesión.
2. Acceder a `/data/importar-ventas`.
3. Revisar respuesta JSON.
4. Verificar registros en PostgreSQL.

Ejemplo de respuesta

```

{
  "creados": 8000,
  "omitidos": 0,
  "errores": 0
}

```

Segunda ejecución esperada

Si se ejecuta nuevamente, debería devolver muchos registros omitidos:

```
{
  "creados": 0,
  "omitidos": 8000,
  "errores": 0
}
```

Mensaje docente

“Este control de duplicados evita cargar dos veces la misma venta.”

BLOQUE 8 — Consultas agregadas tipo OLAP

Tiempo: 45 min

Archivo

```
app/data/services.py
```

Explicación conceptual

OLAP significa análisis multidimensional. En este proyecto se entiende como consultas que agrupan ventas por tiempo, producto, cliente, sucursal o canal.

Preguntas de negocio

Antes de escribir consultas, se formulan preguntas:

- ¿Cuánto se vendió en total?
- ¿Qué sucursal vendió más?
- ¿Qué categoría genera más ingresos?
- ¿Qué canal de venta es más importante?
- ¿Qué productos tienen mayor volumen?
- ¿Qué segmento de cliente compra más?
- ¿Cómo evolucionan las ventas por mes?

8.1 Total de ventas netas

```
def obtener_total_ventas():
    total = db.session.query(func.sum(Venta.total_neto)).scalar()
    return total or 0
```

Explicación

- `func.sum()` suma una columna.
- `.scalar()` devuelve un único valor.
- `or 0` evita mostrar `None` cuando no hay datos.

8.2 Cantidad total vendida

```
def obtener_cantidad_total():
    total = db.session.query(func.sum(Venta.cantidad)).scalar()
    return total or 0
```

8.3 Ticket promedio

```
def obtener_ticket_promedio():
    promedio = db.session.query(func.avg(Venta.total_neto)).scalar()
    return promedio or 0
```

8.4 Ventas por categoría

```
def ventas_por_categoria():
    return db.session.query(
        Venta.categoria_producto.label("categoria"),
        func.sum(Venta.total_neto).label("ventas")
    ).group_by(Venta.categoria_producto)
    .order_by(func.sum(Venta.total_neto).desc())
    .all()
```

8.5 Ventas por sucursal

```
def ventas_por_sucursal():
    return db.session.query(
        Venta.sucursal.label("sucursal"),
        func.sum(Venta.total_neto).label("ventas")
    )
```

```
).group_by(Venta.sucursal)
.order_by(func.sum(Venta.total_neto).desc())
.all()
```

8.6 Ventas por canal

```
def ventas_por_canal():
    return db.session.query(
        Venta.canal_venta.label("canal"),
        func.sum(Venta.total_neto).label("ventas")
    ).group_by(Venta.canal_venta)
    .order_by(func.sum(Venta.total_neto).desc())
    .all()
```

8.7 Top productos por ventas

```
def top_productos_por_ventas(limit=10):
    return db.session.query(
        Venta.nombre_producto.label("producto"),
        func.sum(Venta.total_neto).label("ventas"),
        func.sum(Venta.cantidad).label("unidades")
    ).group_by(Venta.nombre_producto)
    .order_by(func.sum(Venta.total_neto).desc())
    .limit(limit)
    .all()
```

8.8 Ventas por segmento de cliente

```
def ventas_por_segmento_cliente():
    return db.session.query(
        Venta.segmento_cliente.label("segmento"),
        func.sum(Venta.total_neto).label("ventas"),
        func.count(func.distinct(Venta.id_cliente)).label("clientes")
    ).group_by(Venta.segmento_cliente)
    .order_by(func.sum(Venta.total_neto).desc())
    .all()
```

8.9 Ventas por mes

```
def ventas_por_mes():
    return db.session.query(
```

```

        extract("year", Venta.fecha_venta).label("anio"),
        extract("month", Venta.fecha_venta).label("mes"),
        func.sum(Venta.total_neto).label("ventas")
    ).group_by(
        extract("year", Venta.fecha_venta),
        extract("month", Venta.fecha_venta)
    ).order_by(
        extract("year", Venta.fecha_venta),
        extract("month", Venta.fecha_venta)
    ).all()

```

8.10 Cubo OLAP básico

```

def cubo_ventas_sucursal_categoria_mes():
    return db.session.query(
        extract("year", Venta.fecha_venta).label("anio"),
        extract("month", Venta.fecha_venta).label("mes"),
        Venta.sucursal.label("sucursal"),
        Venta.categoria_producto.label("categoria"),
        func.sum(Venta.total_neto).label("ventas"),
        func.sum(Venta.cantidad).label("unidades"),
        func.avg(Venta.total_neto).label("ticket_promedio")
    ).group_by(
        extract("year", Venta.fecha_venta),
        extract("month", Venta.fecha_venta),
        Venta.sucursal,
        Venta.categoria_producto
    ).order_by(
        extract("year", Venta.fecha_venta),
        extract("month", Venta.fecha_venta),
        Venta.sucursal,
        Venta.categoria_producto
    ).all()

```

Explicación del cubo

Este cubo responde preguntas como:

- ¿Cuánto vendió cada sucursal por mes?
- ¿Qué categoría domina en cada sucursal?
- ¿Cómo varía la cantidad vendida según mes y categoría?
- ¿Qué combinación sucursal-categoría tiene mejor ticket promedio?

Mensaje docente

“Una consulta OLAP no solo calcula un total. Agrupa datos desde varias dimensiones para analizar el negocio.”

BLOQUE 9 — Integración mínima con dashboard

Tiempo: 25 min

Archivo

```
app/dashboard/routes.py
```

Objetivo

Mostrar algunos resultados agregados en una vista simple. En la Clase 4 se convertirán en gráficos y tablas interactivas.

Código de referencia

```
from flask import Blueprint, render_template
from flask_login import login_required
from app.data.services import (
    obtener_total_ventas,
    obtener_cantidad_total,
    obtener_ticket_promedio,
    ventas_por_categoria,
    ventas_por_sucursal,
    top_productos_por_ventas
)

dashboard_bp = Blueprint("dashboard", __name__)

@dashboard_bp.route("/")
@login_required
def index():
    total_ventas = obtener_total_ventas()
    cantidad_total = obtener_cantidad_total()
    ticket_promedio = obtener_ticket_promedio()
    categorias = ventas_por_categoria()
    sucursales = ventas_por_sucursal()
    productos = top_productos_por_ventas()
```

```

return render_template(
    "dashboard.html",
    total_ventas=total_ventas,
    cantidad_total=cantidad_total,
    ticket_promedio=ticket_promedio,
    categorias=categorias,
    sucursales=sucursales,
    productos=productos
)

```

Template básico

Archivo: `app/templates/dashboard.html`

```

<h2>Resumen Comercial Café Andino</h2>

<div class="kpi-grid">
  <div class="kpi-card">
    <span>Total ventas netas</span>
    <strong>{{ "{:,.2f}".format(total_ventas) }}</strong>
  </div>
  <div class="kpi-card">
    <span>Unidades vendidas</span>
    <strong>{{ cantidad_total }}</strong>
  </div>
  <div class="kpi-card">
    <span>Ticket promedio</span>
    <strong>{{ "{:,.2f}".format(ticket_promedio) }}</strong>
  </div>
</div>

<h3>Ventas por categoría</h3>
<table>
  <thead>
    <tr>
      <th>Categoría</th>
      <th>Ventas</th>
    </tr>
  </thead>
  <tbody>
    {% for item in categorias %}
    <tr>
      <td>{{ item.categoria }}</td>
      <td>{{ "{:,.2f}".format(item.ventas) }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>

```

```

    </tbody>
</table>

<h3>Top productos</h3>
<table>
  <thead>
    <tr>
      <th>Producto</th>
      <th>Ventas</th>
      <th>Unidades</th>
    </tr>
  </thead>
  <tbody>
    {% for item in productos %}
    <tr>
      <td>{{ item.producto }}</td>
      <td>{{ "{:,.2f}".format(item.ventas) }}</td>
      <td>{{ item.unidades }}</td>
    </tr>
    {% endfor %}
  </tbody>
</table>

```

Qué explicar

- Los datos vienen desde `services.py`.
- La ruta solo coordina.
- El template solo presenta.
- Esta separación hace el proyecto más limpio.

Mensaje docente

“El dashboard no calcula. El dashboard muestra. La lógica vive en servicios.”

BLOQUE 10 — Práctica final, validación y cierre

Tiempo: 20 min

Actividad final obligatoria

Cada estudiante debe lograr: 1. Importar el CSV. 2. Verificar registros en PostgreSQL. 3. Ejecutar la importación dos veces y comprobar duplicados omitidos. 4. Crear al menos 5 consultas agregadas. 5. Mostrar 3 KPIs en una vista. 6. Mostrar una tabla de ventas por categoría. 7. Mostrar una tabla de top productos.

Evidencias solicitadas

- Captura de JSON de importación.
- Captura de tabla `ventas` en PostgreSQL.
- Captura de dashboard con KPIs.
- Captura de consulta agrupada.

Preguntas de cierre

- ¿Por qué usamos `services.py`?
 - ¿Qué hace `group_by`?
 - ¿Qué diferencia hay entre `sum`, `avg` y `count`?
 - ¿Qué problema resuelve `id_venta_csv`?
 - ¿Por qué el dashboard no debe leer el CSV directamente?
-

10. Preguntas de evaluación rápida

1. ¿Qué hace `csv.DictReader`?
 2. ¿Por qué se convierten los datos antes de insertarlos?
 3. ¿Qué función evita duplicados?
 4. ¿Qué representa una fila del CSV dentro del ORM?
 5. ¿Qué hace `func.sum()`?
 6. ¿Qué hace `group_by()`?
 7. ¿Qué diferencia existe entre `.all()` y `.scalar()`?
 8. ¿Qué significa una consulta tipo OLAP?
 9. ¿Por qué se separa la lógica en `services.py`?
 10. ¿Qué ventaja tiene guardar el CSV en PostgreSQL?
-

11. Errores frecuentes y solución rápida

1. UnicodeDecodeError al leer CSV

Usar `encoding="utf-8-sig"`.

2. Error de formato de fecha

Revisar `convertir_fecha()` y agregar el formato correcto.

3. Error al convertir número

Usar `convertir_float()` y reemplazar coma por punto.

4. Registros duplicados

Verificar `id_venta_csv` y `unique=True`.

5. La tabla no tiene datos

Confirmar que la ruta del CSV sea correcta.

6. Error de importación circular

Revisar imports entre `services.py`, `routes.py` y `models.py`.

7. No aparece la ruta `/data/importar-ventas`

Verificar que `data_bp` esté registrado en `app/__init__.py`.

8. `.scalar()` devuelve None

No hay datos cargados o la columna está vacía. Usar `or 0`.

12. Archivos finales esperados en Clase 3

`app/data/services.py`

Debe contener:

- funciones de limpieza.
- función de importación CSV.
- consultas agregadas.
- consultas OLAP básicas.

`app/data/routes.py`

Debe contener:

- Blueprint `data_bp`.
- ruta protegida de importación.

`app/dashboard/routes.py`

Debe contener:

- llamadas a funciones de servicios.
- envío de datos al template.

`app/templates/dashboard.html`

Debe contener:

- KPIs.
- tablas simples con resultados agregados.

13. Producto final de Clase 3

El proyecto al finalizar esta clase debe ser la suma de:

Clase 1: Flask + login + layout base
Clase 2: PostgreSQL + ORM + migraciones
Clase 3: CSV + services + consultas OLAP

Resultado técnico

- login funcional.
- base de datos activa.
- tabla `usuarios` funcionando.
- tabla `ventas` poblada.
- importación CSV protegida.
- consultas agregadas funcionando.
- datos mostrados en dashboard simple.

Resultado pedagógico

El estudiante entiende cómo un sistema web pasa de datos crudos a información comercial.

14. Mensaje de cierre docente

“Hoy dimos el paso más importante para la analítica: convertir un archivo CSV en una base de datos consultable. Ya no tenemos solo una aplicación web; ahora tenemos un sistema con

datos reales y consultas comerciales. En la Clase 4 convertiremos estas consultas en gráficos, tablas interactivas y dashboards SaaS profesionales.”

15. Recomendación docente

No avanzar a Chart.js ni DataTables si los estudiantes no comprenden:

- lectura del CSV.
- creación de objetos `Venta`.
- `db.session.add()`.
- `db.session.commit()`.
- control de duplicados.
- `func.sum()`.
- `group_by()`.
- separación entre ruta y servicio. La Clase 4 depende directamente de estas consultas.